



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

INTELIGENCIA ARTIFICIAL

EJERCICIOS DE PROCESAMIENTO DE LENGUAJE NATURAL (NLP)

JUAN FERNANDO PEREZ OLAYA
NELSON DAVID GUTIERREZ MEDINA
OSCAR FABIAN LEON TRIANA
SNEIDER MONROY QUIROGA
WILKYN JULIAN VARGAS BAHAMON

CORHUILA

DOCENTE: JOSE MIGUEL LLANOS MOSQUERA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

CORPORACION UNIVERSITARIA DEL HUILA CORHUILA
ING. EN SISTEMAS
NEIVA - HUILA
2026





CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Tabla de Contenido

Tabla de Contenido	2
Lista de Figuras	3
1 Introducción	4
2 Objetivos Generales	5
3 Objetivos Específicos	6
4 Marco Teórico	7
5 Desarrollo	8
5.1 Limpieza y Preprocesamiento Básico	8
5.2 Tokenización	10
5.3 Normalización de Texto	12
5.4 Remoción de Stopwords	14
5.5 Stemming vs Lematización	16
5.6 Análisis de Frecuencia de Palabras	18
5.7 Extracción de Entidades Nombradas (NER)	20
5.8 Similaridad entre Textos	22
5.9 Pipeline Completo de Preprocesamiento	24
5.10 Proyecto Integrador - Análisis de Sentimientos Básico	27
Conclusiones	30
Bibliografía	31

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación





CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Lista de Figuras

Figura 1 Código del Ejercicio 1	8
Figura 2 Resultado del Ejercicio 1	9
Figura 3 Código del Ejercicio 2	10
Figura 4 Resultado del Ejercicio 2	11
Figura 5 Código del Ejercicio 3	12
Figura 6 Resultado del Ejercicio 3	13
Figura 7 Código del Ejercicio 4	14
Figura 8 Resultado del Ejercicio 4	15
Figura 9 Código del Ejercicio 5	16
Figura 10 Resultado del Ejercicio 5	17
Figura 11 Código del Ejercicio 6	18
Figura 12 Resultado del Ejercicio 6	19
Figura 13 Código del Ejercicio 7 Parte 1.....	20
Figura 14 Código del Ejercicio 7 Parte 2.....	21
Figura 15 Resultado del Ejercicio 7	21
Figura 16 Código del Ejercicio 8	22
Figura 17 Resultado del Ejercicio 8	23
Figura 18 Código del Ejercicio 9 Parte 1.....	24
Figura 19 Código del Ejercicio 9 Parte 2.....	25
Figura 20 Código del Ejercicio 9 Parte 3.....	25
Figura 21 Resultado del Ejercicio 9	26
Figura 22 Código del Ejercicio 10 Parte 1.....	27
Figura 23 Código del Ejercicio 10 Parte 2.....	28
Figura 24 Código del Ejercicio 10 Parte 3.....	29
Figura 25 Resultado del Ejercicio 10	29

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación





1 Introducción

En el presente trabajo se desarrolla un taller enfocado en el área de Procesamiento del Lenguaje Natural (NLP), cuyo propósito principal es aplicar diversas técnicas fundamentales para el análisis y manipulación de textos. A lo largo del documento, se abordan actividades prácticas que permiten comprender cómo las máquinas pueden procesar, interpretar y extraer información del lenguaje humano.

El desarrollo del taller incluye la implementación de procesos esenciales como la limpieza y preprocesamiento de texto, tokenización, normalización, eliminación de palabras vacías (stopwords), así como técnicas más avanzadas como stemming, lematización, análisis de frecuencia, extracción de entidades nombradas (NER) y cálculo de similitud entre textos. Además, se integra todo este conocimiento en la construcción de un pipeline completo de procesamiento y un proyecto final orientado al análisis de sentimientos.

Cada ejercicio ha sido diseñado para fortalecer las habilidades prácticas en programación y análisis de datos textuales, utilizando herramientas ampliamente empleadas en NLP como NLTK, spaCy y scikit-learn. De esta manera, se busca no solo comprender los conceptos teóricos, sino también aplicarlos en contextos reales, permitiendo evidenciar el potencial de estas técnicas en diferentes aplicaciones.

CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

2 Objetivos Generales

Desarrollar e implementar un conjunto de técnicas fundamentales de Procesamiento del Lenguaje Natural (NLP) que permitan el análisis, procesamiento e interpretación de textos, mediante la construcción de un pipeline completo y su aplicación en un proyecto integrador de análisis de sentimientos.



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación





3 Objetivos Específicos

- Implementar procesos de limpieza y preprocesamiento de texto para eliminar información irrelevante como URLs, correos electrónicos y caracteres especiales.
- Aplicar técnicas de tokenización para dividir los textos en unidades básicas como palabras y oraciones.
- Desarrollar métodos de normalización de texto, incluyendo la eliminación de acentos y estandarización de caracteres.
- Identificar y eliminar palabras vacías (stopwords) para mejorar la calidad del análisis textual.
- Comparar técnicas de reducción de palabras como stemming y lematización, analizando sus ventajas y desventajas.
- Realizar análisis de frecuencia para identificar las palabras más relevantes dentro de un conjunto de textos.
- Implementar la extracción de entidades nombradas (NER) para reconocer información clave como personas, lugares y organizaciones.
- Calcular la similitud entre textos utilizando técnicas como TF-IDF y similitud coseno.
- Diseñar e implementar un pipeline completo de procesamiento de texto que integre todas las etapas anteriores.
- Aplicar las técnicas desarrolladas en un proyecto práctico de análisis de sentimientos para clasificar textos según su polaridad.

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

4 Marco Teórico

El Procesamiento del Lenguaje Natural (NLP, por sus siglas en inglés) es una rama de la inteligencia artificial que se enfoca en la interacción entre las computadoras y el lenguaje humano. Su objetivo principal es permitir que las máquinas puedan comprender, interpretar y generar texto de manera similar a como lo hacen los seres humanos.

Uno de los primeros pasos en NLP es el **preprocesamiento de texto**, el cual consiste en limpiar y preparar los datos textuales para su análisis. Esto incluye la eliminación de elementos irrelevantes como enlaces, correos electrónicos y caracteres especiales, así como la normalización del texto mediante la conversión a minúsculas y la estandarización de espacios.

Posteriormente, se aplica la **tokenización**, que permite dividir el texto en unidades más pequeñas llamadas tokens, como palabras o frases. Este proceso es fundamental para facilitar el análisis posterior. Asimismo, la **normalización** busca reducir la variabilidad del lenguaje, eliminando acentos y caracteres diacríticos que puedan afectar la interpretación de los datos.

Otro paso importante es la eliminación de **stopwords**, que son palabras comunes (como artículos y preposiciones) que no aportan un significado relevante al análisis. Esto permite centrarse en los términos más significativos del texto.

En cuanto a la reducción de palabras, existen técnicas como el **stemming**, que recorta las palabras a su raíz básica, y la **lematización**, que transforma las palabras a su forma canónica o lema. Aunque ambas técnicas buscan simplificar el texto, la lematización suele ser más precisa, mientras que el stemming es más rápido pero menos exacto.

El **análisis de frecuencia de palabras** permite identificar los términos más utilizados dentro de un texto, lo cual es útil para detectar temas relevantes. Por otro lado, la **extracción de entidades nombradas (NER)** permite reconocer elementos clave como nombres de personas, organizaciones y lugares, facilitando la estructuración de la información.

Además, el NLP emplea técnicas para medir la relación entre textos, como el modelo **TF-IDF (Term Frequency - Inverse Document Frequency)** y la **similitud coseno**, que permiten cuantificar qué tan similares son dos documentos en función de sus palabras.

Finalmente, la integración de todas estas técnicas en un **pipeline de procesamiento** permite automatizar el análisis de texto de manera eficiente. Esto es especialmente útil en aplicaciones como el **análisis de sentimientos**, donde se busca clasificar opiniones como positivas, negativas o neutrales, siendo una de las aplicaciones más comunes del NLP en contextos reales.

5 Desarrollo

5.1 Limpieza y Preprocesamiento Básico

Objetivo: Crear una función que limpie texto removiendo URLs, emails, caracteres especiales y normalizando espacios.

Instrucciones:

- Convierte a minúsculas
- Remueve URLs (http, https, www)
- Remueve emails
- Remueve caracteres especiales manteniendo solo letras y espacios
- Normaliza espacios en blanco

```
import re

def limpiar_texto(texto):
    """
    Limpia un texto removiendo elementos no deseados.

    Args:
        texto: String con el texto a limpiar

    Returns:
        String con el texto limpio
    """
    texto=texto.lower()
    texto = re.sub(r'http\S+|www\S+', '', texto)
    texto = re.sub(r'\S+@\S+', '', texto)
    texto = re.sub(r'^a-zAÉÍÓÚÑ\s]', '', texto)
    texto = ' '.join(texto.split())
    return texto

# Casos de prueba
textos_prueba = [
    "¡Hola!!! Visita https://ejemplo.com o envía tu CV a recurso@empresa.com. Hay espacios extra!!!",
    "Python3.9 es excelente!!! Descarga desde www.python.org #programming @user",
    "El NLP es fascinante... ¿No lo crees? (sí/no) [responder]"
]

print("Resultados de limpieza:")
print("=" * 60)
for i, texto in enumerate(textos_prueba, 1):
    print(f"\nTexto {i}:")
    print(f"Original: {texto}")
    print(f"Limpio: {limpiar_texto(texto)}")
```

Figura 1 Código del Ejercicio 1



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Resultados de limpieza:

=====

Texto 1:

Original: ¡Hola!!! Visita <https://ejemplo.com> o envía tu CV a recurso@empresa.com. Hay espacios extra!!!

Limpio: hola visita o envía tu cv a hay espacios extra

Texto 2:

Original: Python3.9 es excelente!!! Descarga desde www.python.org #programming @user

Limpio: python es excelente descarga desde programming user

Texto 3:

Original: El NLP es fascinante... ¿No lo crees? (sí/no) [responder]

Limpio: el nlp es fascinante no lo crees sino responder

Figura 2 Resultado del Ejercicio 1

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación



5.2 Tokenización

Objetivo: Dividir textos en tokens (palabras u oraciones) utilizando NLTK.

Instrucciones:

- Descarga los recursos necesarios de NLTK
- Implementa Tokenización de palabras y oraciones
- Compara los resultados para diferentes textos

```
import nltk

# Descargar recursos necesarios
nltk.download('punkt_tab')
nltk.download('punkt')

from nltk.tokenize import word_tokenize, sent_tokenize

def analizar_tokenizacion(texto, idioma='spanish'):
    """
    Realiza tokenización de un texto en palabras y oraciones.

    Args:
        texto: String con el texto a tokenizar
        idioma: Idioma del texto

    Returns:
        Diccionario con tokens de palabras y oraciones
    """
    tokens_palabra = word_tokenize(texto, language=idioma)
    tokens_oracion = sent_tokenize(texto, language=idioma)

    resultado = {
        'tokens_palabra': tokens_palabra,
        'tokens_oracion': tokens_oracion,
        'num_palabras': len(tokens_palabra),
        'num_oraciones': len(tokens_oracion)
    }
    return resultado

# Casos de prueba
texto1 = "Hola. ¿Cómo estás? Estoy muy bien, gracias por preguntar."
texto2 = "El procesamiento del lenguaje natural es increíble! ¿No crees? Sí, totalmente."

for i, texto in enumerate([texto1, texto2], 1):
    print(f"\nTexto {i}: {texto}")
    resultado = analizar_tokenizacion(texto)
    print("Tokens de palabras:", resultado['tokens_palabra'])
    print("Tokens de oraciones:", resultado['tokens_oracion'])
    print("Número de palabras:", resultado['num_palabras'])
    print("Número de oraciones:", resultado['num_oraciones'])
```

Figura 3 Código del Ejercicio 2



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.

Texto 1: Hola. ¿Cómo estás? Estoy muy bien, gracias por preguntar.
Tokens de palabras: ['Hola', '.', '¿Cómo', 'estás', '?', 'Estoy', 'muy', 'bien', ',', 'gracias', 'por', 'preguntar', '.']
Tokens de oraciones: ['Hola.', '¿Cómo estás?', 'Estoy muy bien, gracias por preguntar.']
Número de palabras: 13
Número de oraciones: 3

Texto 2: El procesamiento del lenguaje natural es increíble! ¿No crees? Sí, totalmente.
Tokens de palabras: ['El', 'procesamiento', 'del', 'lenguaje', 'natural', 'es', 'increíble', '!', '¿No', 'crees', '?', 'Sí', ',', 'totalmente', '.']
Tokens de oraciones: ['El procesamiento del lenguaje natural es increíble!', '¿No crees?', 'Sí, totalmente.']
Número de palabras: 15
Número de oraciones: 3
```

Figura 4 Resultado del Ejercicio 2



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

5.3 Normalización de Texto

Objetivo: Normalizar texto removiendo acentos y caracteres

Instrucciones:

- Implementa función que remueva acentos usando unicodedata
- Convierte textos con acentos a su forma normalizada
- Compara el antes y después

```
import unicodedata

def normalizar_texto(texto):
    """
    Normaliza un texto removiendo acentos y caracteres diacríticos.

    Args:
        texto: String con el texto a normalizar

    Returns:
        String normalizado
    """

    texto = texto.lower() # Convertimos todo el texto a minúsculas

    texto_normalizado = unicodedata.normalize('NFD', texto)
    # Normalizamos el texto usando NFD:
    # Esto separa cada letra de su acento (ej: "á" → "a" + " ")

    texto_sin_acentos = ''.join( # Reconstruimos el texto sin acentos
        char for char in texto_normalizado # Recorremos cada carácter del texto normalizado
        if unicodedata.category(char) != 'Mn'
        # Filtramos: eliminamos los caracteres cuya categoría sea 'Mn'
        # 'Mn' significa "Mark, Nonspacing" → es decir, acentos y diacríticos
    )

    return texto_sin_acentos # Retornamos el texto ya limpio sin acentos

# Casos de prueba
textos_acentuados = [
    "¡Hola! ¿Cómo Estás?",
    "Programación y árboles de decisión",
    "La física y química de la Tierra",
    "Café con leche es delicioso"
]

print("Normalización de textos acentuados:")
print("=" * 60)

for texto in textos_acentuados:
    print(f"Original: {texto}")
    print(f"Normalizado: {normalizar_texto(texto)}")
    print()
```

Figura 5 Código del Ejercicio 3



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Normalización de textos acentuados:

Original: ¡Hola! ¿Cómo Estás?

Normalizado: ¡hola! ¿como estas?

Original: Programación y árboles de decisión

Normalizado: programacion y arboles de decision

Original: La física y química de la Tierra

Normalizado: la fisica y quimica de la tierra

Original: Café con leche es delicioso

Normalizado: cafe con leche es delicioso

Figura 6 Resultado del Ejercicio 3

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación



5.4 Remoción de Stopwords

Objetivo: Remover palabras comunes que no aportan significado.

Instrucciones:

- Descarga la lista de stopwords en español
- Implementa función que filtre stopwords
- Analiza cómo cambia un texto después de remover stopwords

```
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords

def remover_stopwords(texto, idioma='spanish'):
    """
    Remueve stopwords de un texto.

    Args:
        texto: String con el texto
        idioma: Idioma para seleccionar stopwords

    Returns:
        Lista de palabras sin stopwords
    """
    # Obtener stopwords en español
    stop_words = set(stopwords.words('spanish'))

    # Tokenizar el texto
    tokens = texto.split()
    tokens_filtrados = [
        token for token in tokens
        if token not in stop_words
    ]

    return tokens_filtrados

# Ejemplos de uso
textos = [
    "El procesamiento del lenguaje natural es fascinante y útil",
    "La inteligencia artificial está transformando el mundo",
    "Los algoritmos de aprendizaje automático son muy poderosos"
]

print("Remoción de Stopwords:")
print("=" * 60)
for texto in textos:
    resultado = remover_stopwords(texto)
    print(f"Original: {texto}")
    print(f"Sin stopwords: {' '.join(resultado)}")
    print(f"Palabras removidas: {len(texto.split()) - len(resultado)}")
    print()
```

Figura 7 Código del Ejercicio 4



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Remoción de Stopwords:

```
=====
Original:      El procesamiento del lenguaje natural es fascinante y útil
Sin stopwords: El procesamiento lenguaje natural fascinante útil
Palabras removidas: 3

Original:      La inteligencia artificial está transformando el mundo
Sin stopwords: La inteligencia artificial transformando mundo
Palabras removidas: 2

Original:      Los algoritmos de aprendizaje automático son muy poderosos
Sin stopwords: Los algoritmos aprendizaje automático poderosos
Palabras removidas: 3

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
```

Figura 8 Resultado del Ejercicio 4

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación



5.5 Stemming vs Lemmatización

Objetivo: Comparar dos técnicas de reducción de palabras.

Instrucciones:

- Implementa stemming usando SnowballStemmer
- Implementa lematización
- Compara resultados para las mismas palabras
- Analiza ventajas y desventajas de cada enfoque

```
from nltk.stem.snowball import SnowballStemmer
from nltk.stem import WordNetLemmatizer
import nltk

nltk.download('wordnet')
nltk.download('omw-1.4')

def comparar_stemming_lemmatizacion(palabras):
    """
    Compara stemming y lematización para un conjunto de palabras.

    Args:
        palabras: Lista de palabras a procesar

    Returns:
        Diccionario con resultados
    """
    # Crear instancias
    stemmer = SnowballStemmer('english')
    lemmatizer = WordNetLemmatizer()

    # Palabras en español para stemming
    palabras_es = [
        'correr', 'corriendo', 'corrió', 'corrida',
        'programar', 'programación', 'programador',
        'análisis', 'analizar', 'analizando'
    ]

    print("Comparación: Stemming (Español)")
    print("=" * 60)
    stemmer = SnowballStemmer('spanish')
    for palabra in palabras_es:
        stem = stemmer.stem(palabra)
        lema = lemmatizer.lemmatize(palabra, pos='v')

        print(f"Palabra: {palabra:15} | Stem: {stem:10} | Lema: {lema}")

    # Palabras en inglés para lematización
    palabras_en = [
        'running', 'runs', 'ran',
        'programming', 'programs', 'programmed',
        'analyzing', 'analyzed', 'analyzes'
    ]

    print("\nComparación: Lemmatización (Inglés)")
    print("=" * 60)
    lemmatizer = WordNetLemmatizer()
    for palabra in palabras_en:
        stem = stemmer.stem(palabra)
        lema = lemmatizer.lemmatize(palabra, pos='v')

        print(f"Palabra: {palabra:15} | Stem: {stem:10} | Lema: {lema}")
```

Figura 9 Código del Ejercicio 5



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Comparación: Stemming (Español)

Palabra: correr	Stem: corr	Lema: correr
Palabra: corriendo	Stem: corr	Lema: corriendo
Palabra: corrió	Stem: corr	Lema: corrió
Palabra: corrida	Stem: corr	Lema: corrida
Palabra: programar	Stem: program	Lema: programar
Palabra: programación	Stem: program	Lema: programación
Palabra: programador	Stem: program	Lema: programador
Palabra: análisis	Stem: analisis	Lema: análisis
Palabra: analizar	Stem: analiz	Lema: analizar
Palabra: analizando	Stem: analiz	Lema: analizando

Comparación: Lemmatización (Inglés)

Palabra: running	Stem: running	Lema: run
Palabra: runs	Stem: runs	Lema: run
Palabra: ran	Stem: ran	Lema: run
Palabra: programming	Stem: programming	Lema: program
Palabra: programs	Stem: programs	Lema: program
Palabra: programmed	Stem: programm	Lema: program
Palabra: analyzing	Stem: analyzing	Lema: analyze
Palabra: analyzed	Stem: analyz	Lema: analyze
Palabra: analyzes	Stem: analyz	Lema: analyze
[nltk_data] Downloading package wordnet to /root/nltk_data...		
[nltk_data] Package wordnet is already up-to-date!		
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...		
[nltk_data] Package omw-1.4 is already up-to-date!		

Figura 10 Resultado del Ejercicio 5

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación



5.6 Análisis de Frecuencia de Palabras

Objetivo: Identificar las palabras más frecuentes en un texto.

Instrucciones:

- Implementa función que cuente frecuencia de palabras
- Remueve stopwords antes de contar
- Encuentra las N palabras más comunes
- Visualiza los resultados

```
from collections import Counter
from nltk.corpus import stopwords

def obtener_palabras_clave(texto, n=10, idioma='spanish'):
    """
    Extrae las N palabras más frecuentes de un texto.

    Args:
        texto: String con el texto a analizar
        n: Número de palabras clave a retornar
        idioma: Idioma del texto

    Returns:
        Lista de tuplas (palabra, frecuencia)
    """

    # 1. Obtener stopwords
    stop_words = set(stopwords.words(idioma))

    # 2. Limpiar y tokenizar texto
    texto = texto.lower()
    palabras = texto.split()

    # 3. Filtrar stopwords y palabras no alfabéticas
    palabras_filtradas = [
        palabra for palabra in palabras
        if palabra.isalpha() and palabra not in stop_words
    ]

    # 4. Contar frecuencias
    frecuencias = Counter(palabras_filtradas)

    # 5. Obtener top N
    return frecuencias.most_common(n)

# Texto de ejemplo
texto_analisis = """
La Inteligencia Artificial es revolucionaria. La IA transforma industrias.
El aprendizaje automático es clave. Los modelos de IA mejoran constantemente.
Los algoritmos de machine learning procesan datos masivos.
La IA encontrada en aplicaciones modernas es muy sofisticada.
Programación y IA son futuro del software.
"""

print("Análisis de Palabras Clave")
print("=" * 60)

resultado = obtener_palabras_clave(texto_analisis, n=10)

total_palabras = len(texto_analisis.split())

print(f"Total de palabras en el texto: {total_palabras}\n")

print(f"{'Palabra':<20}{'Frecuencia':<12}{'Porcentaje':<10}")
print(f"{'-' * 45}")

# Iterar e imprimir tabla
for palabra, frecuencia in resultado:
    porcentaje = (frecuencia / total_palabras) * 100
    print(f"{'palabra':<20}{'frecuencia':<12}{'porcentaje':.2f}%")
```

Figura 11 Código del Ejercicio 6



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Análisis de Palabras Clave

Total de palabras en el texto: 44

Palabra	Frecuencia	Porcentaje
ia	4	9.09%
inteligencia	1	2.27%
artificial	1	2.27%
transforma	1	2.27%
aprendizaje	1	2.27%
automático	1	2.27%
modelos	1	2.27%
mejoran	1	2.27%
algoritmos	1	2.27%
machine	1	2.27%

Figura 12 Resultado del Ejercicio 6

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación



5.7 Extracción de Entidades Nombradas (NER)

Objetivo: Identificar entidades como personas, lugares y organizaciones.

Instrucciones:

- Descarga el modelo de spaCy en español
- Procesa textos para extraer entidades
- Clasifica entidades por tipo
- Analiza la información extraída

```
import spacy
import subprocess
import sys

subprocess.check_call([sys.executable, "-m", "pip", "install", "spacy", "-q"])
subprocess.check_call([sys.executable, "-m", "spacy", "download", "es_core_news_sm", "-q"])

import spacy

def extraer_entidades(texto):
    """
    Extrae entidades nombradas de un texto.

    Args:
        texto: String con el texto a procesar

    Returns:
        Lista de diccionarios con entidades
    """
    # 1. Cargar modelo de spaCy
    nlp = spacy.load("es_core_news_sm")

    # 2. Procesar texto
    doc = nlp(texto)

    # 3. Extraer entidades
    entidades = []
    for ent in doc.ents:
        entidades.append({
            "entidad": ent.text,
            "tipo": ent.label_
        })

    # 4. Retornar lista
    return entidades
pass

# Textos de ejemplo
textos_ner = [
    """
    Juan García, director ejecutivo de Google, anunció hoy en Bogotá
    la apertura de una oficina regional. La inversión será de 50 millones
    de dólares en los próximos 3 años.
    """,
    """
    María López trabaja en Microsoft desde el 2020. Ella es ingeniera
    de software y reside en Seattle, Washington. Su proyecto actual
    es sobre inteligencia artificial.
    """
]
```

Figura 13 Código del Ejercicio 7 Parte 1



```
print("Extracción de Entidades (NER)")
print("=" * 70)

for i, texto in enumerate(textos_ner, 1):
    print(f"\nTexto {i}:")
    print(f"{texto.strip()}")
    print(f"\nEntidades identificadas:")
    entidades = extraer_entidades(texto)

    for ent in entidades:
        # Clasificación básica del tipo
        if ent["tipo"] == "PER":
            descripcion = "Persona"
        elif ent["tipo"] == "ORG":
            descripcion = "Organización"
        elif ent["tipo"] in ["LOC", "GPE"]:
            descripcion = "Lugar"
        elif ent["tipo"] == "DATE":
            descripcion = "Fecha"
        elif ent["tipo"] == "MONEY":
            descripcion = "Dinero"
        else:
            descripcion = "Otro"

    print(f'- Entidad: "{ent["entidad"]}", Tipo: {ent["tipo"]}, Descripción: "{descripcion}"')
```

Figura 14 Código del Ejercicio 7 Parte 2

```
Extracción de Entidades (NER)
=====

Texto 1:
Juan García, director ejecutivo de Google, anunció hoy en Bogotá
la apertura de una oficina regional. La inversión será de 50 millones
de dólares en los próximos 3 años.

Entidades identificadas:
- Entidad: "Juan García", Tipo: PER, Descripción: "Persona"
- Entidad: "Google", Tipo: ORG, Descripción: "Organización"
- Entidad: "Bogotá", Tipo: LOC, Descripción: "Lugar"
- Entidad: "La inversión será", Tipo: MISC, Descripción: "Otro"

Texto 2:
María López trabaja en Microsoft desde el 2020. Ella es ingeniera
de software y reside en Seattle, Washington. Su proyecto actual
es sobre inteligencia artificial.

Entidades identificadas:
- Entidad: "Microsoft", Tipo: ORG, Descripción: "Organización"
- Entidad: "Seattle", Tipo: LOC, Descripción: "Lugar"
- Entidad: "Washington", Tipo: LOC, Descripción: "Lugar"
```

Figura 15 Resultado del Ejercicio 7

5.8 Similitud entre Textos

Objetivo: Calcular qué tan similares son dos textos usando similitud coseno.

Instrucciones:

- Implementa función para calcular similitud con TF-IDF y cosine_similarity
- Compara diferentes pares de textos
- Interpreta los resultados (0 = nada similar, 1 = idéntico)

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

def calcular_similitud_textos(textos):
    """
    Calcula matriz de similitud entre textos usando TF-IDF.

    Args:
        textos: Lista de textos a comparar

    Returns:
        Matriz de similitud (numpy array)
    """

    vectorizador = TfidfVectorizer()
    # Creamos el vectorizador TF-IDF
    # Este convierte cada texto en un vector numérico basado en importancia de palabras

    matriz_tfidf = vectorizador.fit_transform(textos)
    # Ajusta el modelo a los textos (aprende vocabulario)
    # y transforma los textos en una matriz TF-IDF

    matriz_similitud = cosine_similarity(matriz_tfidf)
    # Calcula la similitud coseno entre todos los textos
    # Resultado: matriz donde cada valor indica qué tan similares son dos textos

    return matriz_similitud # Retornamos la matriz de similitud

# Textos de prueba
textos_similitud = [
    "El procesamiento del lenguaje natural es importante para la IA",
    "NLP y procesamiento de texto son relevantes en inteligencia artificial",
    "Los gatos son animales domésticos muy populares",
    "La comprensión del lenguaje natural mediante máquinas es fundamental",
    "Los perros y gatos son mascotas comunes en los hogares"
]

print("Análisis de Similitud entre Textos")
print("=" * 80)
print("\nTextos a comparar:")
for i, texto in enumerate(textos_similitud, 1):
    print(f"{i}. {texto}")

print("\n" + "=" * 80)
print("Matriz de Similitud (Similitud Coseno):")
print("=" * 80)

# Llamamos a la función para obtener la matriz de similitud
matriz = calcular_similitud_textos(textos_similitud)

# Redondeamos los valores a 2 decimales para que sea más legible
matriz_redondeada = np.round(matriz, 2)

# Imprimimos la matriz completa
print(matriz_redondeada)

print("\nInterpretación:")
print("- Valores cercanos a 1 → textos muy similares")
print("- Valores cercanos a 0 → textos poco o nada similares")
```

Figura 16 Código del Ejercicio 8



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Análisis de Similitud entre Textos

Textos a comparar:

1. El procesamiento del lenguaje natural es importante para la IA
2. NLP y procesamiento de texto son relevantes en inteligencia artificial
3. Los gatos son animales domésticos muy populares
4. La comprensión del lenguaje natural mediante máquinas es fundamental
5. Los perros y gatos son mascotas comunes en los hogares

Matriz de Similitud (Similitud Coseno):

```
[[1. 0.08 0. 0.43 0. ]  
 [0.08 1. 0.07 0. 0.14]  
 [0. 0.07 1. 0. 0.35]  
 [0.43 0. 0. 1. 0. ]  
 [0. 0.14 0.35 0. 1. ]]
```

Interpretación:

- Valores cercanos a 1 → textos muy similares
- Valores cercanos a 0 → textos poco o nada similares

Figura 17 Resultado del Ejercicio 8

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

5.9 Pipeline Completo de Preprocesamiento

Objetivo: Crear una clase que integre todas las técnicas anteriores.

Instrucciones:

- Implementa clase PreprocesadorTexto con métodos para cada paso
- Integra: limpieza, tokenización, normalización, stopwords, stemming
- Crea método procesar() que execute el pipeline completo
- Prueba con varios textos complejos

```
import re
import unicodedata
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem.snowball import SnowballStemmer

nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)

class PreprocesadorTexto:
    """
    Pipeline completo de preprocesamiento de texto en español.

    Pasos:
    1. Limpieza (URLs, emails, caracteres especiales)
    2. Normalización (acentos)
    3. Tokenización
    4. Remoción de stopwords
    5. Stemming
    """

    def __init__(self, idioma='spanish'):
        self.idioma = idioma
        self.stemmer = SnowballStemmer(idioma)
        self.stopwords_set = set(stopwords.words(idioma))

    def _limpiar(self, texto):
        """Limpia el texto eliminando URLs, emails y caracteres especiales."""
        texto = re.sub(r"http\S+|www\S+|https\S+", " ", texto) # URLs
        texto = re.sub(r"\S+@\S+", " ", texto) # Emails
        texto = re.sub(r"^[a-zA-ZáéíóúÁÉÍÓÚÑ\s]", " ", texto) # Caracteres especiales
        return texto

    def _normalizar(self, texto):
        """Normaliza acentos y convierte a minúsculas."""
        texto = texto.lower()
        texto = unicodedata.normalize("NFD", texto)
        texto = texto.encode("ascii", "ignore").decode("utf-8")
        return texto
```

Figura 18 Código del Ejercicio 9 Parte 1



```
def _tokenizar(self, texto):
    """Tokeniza el texto en palabras."""
    return word_tokenize(texto, language=self.idioma)

def _remover_stopwords(self, tokens):
    """Remueve stopwords."""
    return [t for t in tokens if t not in self.stopwords_set and len(t) > 2]

def _aplicar_stemming(self, tokens):
    """Aplica stemming."""
    return [self.stemmer.stem(t) for t in tokens]

def procesar(self, texto, verbose=False):
    """Ejecuta el pipeline completo."""
    if verbose:
        print(f"Texto original:\n{texto[:200]}...\n")

    texto = self._limpiar(texto)
    if verbose:
        print(f"Después de limpieza:\n{texto[:200]}...\n")

    texto = self._normalizar(texto)
    if verbose:
        print(f"Después de normalización:\n{texto[:200]}...\n")

    tokens = self._tokenizar(texto)
    if verbose:
        print(f"Tokens iniciales ({len(tokens)}):\n{tokens[:20]}\n")

    tokens = self._remover_stopwords(tokens)
    if verbose:
        print(f"Tokens sin stopwords ({len(tokens)}):\n{tokens[:20]}\n")

    tokens = self._aplicar_stemming(tokens)
    if verbose:
        print(f"Tokens finales ({len(tokens)}):\n{tokens[:20]}\n")

    return tokens
```

Figura 19 Código del Ejercicio 9 Parte 2

```
# Casos de uso
textos_complejos = [
    """
    ¡Hola a todos! Este es un ejemplo AVANZADO de procesamiento del lenguaje natural.
    Visita https://ejemplo.com o escribe a contacto@empresa.com para más información.
    El NLP es fascinante... ¿No lo crees? #AI #IA @usuario
    """,
    """
    La inteligencia artificial (IA) está revolucionando el mundo!!!
    Los algoritmos de Machine Learning procesan MILLONES de datos.
    La programación en Python3.9+ es excelente (10/10). ¿Verdad???
    """
]

print("Pipeline Completo de Preprocesamiento")
print("=" * 80)

for i, texto in enumerate(textos_complejos, 1):
    print(f"\n\n{'='*80}")
    print(f"EJEMPLO {i}")
    print(f"{'='*80}")

    preprocesador = PreprocesadorTexto()
    tokens_finales = preprocesador.procesar(texto, verbose=True)
```

Figura 20 Código del Ejercicio 9 Parte 3



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

```

Visita https://ejemplo.com o escribe a contacto@empresa.com para más informacón.
El NLP es fascinante.....

Después de limpieza:

HOLA a todos Este es un ejemplo AVANZADO de procesamiento del lenguaje natural
Visita o escribe a para más informacón
El NLP es fascinante No lo crees AI IA usuario
...

Después de normalización:

hola a todos este es un ejemplo avanzado de procesamiento del lenguaje natural
visita o escribe a para mas informacion
el nlp es fascinante no lo crees ai ia usuario
...

Tokens iniciales (30):
['hola', 'a', 'todos', 'este', 'es', 'un', 'ejemplo', 'avanzado', 'de', 'procesamiento', 'del', 'lenguaje', 'natural', 'visita', 'o', 'escribe', 'a', 'para', 'mas', 'informacion']

Tokens sin stopwords (14):
['hola', 'ejemplo', 'avanzado', 'procesamiento', 'lenguaje', 'natural', 'visita', 'escribe', 'mas', 'informacion', 'nlp', 'fascinante', 'crees', 'usuario']

Tokens finales (14):
['hol', 'ejempl', 'avanz', 'proces', 'lenguaj', 'natural', 'visit', 'escrib', 'mas', 'inform', 'nlp', 'fascin', 'cre', 'usuari']

=====
EJEMPLO 2
=====
Texto original:

La inteligencia artificial (IA) está revolucionando el mundo!!!
Los algoritmos de Machine Learning procesan MILLONES de datos.
La programación en Python3.9+ es excelente (10/10). ¿Verdad?...

Después de limpieza:

La inteligencia artificial IA está revolucionando el mundo
Los algoritmos de Machine Learning procesan MILLONES de datos
La programación en Python es excelente Verdad ...

Después de normalización:

la inteligencia artificial ia esta revolucionando el mundo
los algoritmos de machine learning procesan millones de datos
la programacion en python es excelente verdad ...

Tokens iniciales (24):
['la', 'inteligencia', 'artificial', 'ia', 'esta', 'revolucionando', 'el', 'mundo', 'los', 'algoritmos', 'de', 'machine', 'learning', 'procesan', 'millones', 'de', 'datos', 'la', 'programacion', 'en']

Tokens sin stopwords (14):
['inteligencia', 'artificial', 'revolucionando', 'mundo', 'algoritmos', 'machine', 'learning', 'procesan', 'millones', 'datos', 'programacion', 'python', 'excelente', 'verdad']

Tokens finales (14):
['inteligent', 'artificial', 'revolucion', 'mund', 'algoritm', 'machin', 'learning', 'proces', 'millon', 'dat', 'program', 'python', 'excelent', 'verd']

```

Figura 21 Resultado del Ejercicio 9

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

5.10 Proyecto Integrador - Análisis de Sentimientos Básico

Objetivo: Integrar todas las técnicas para un proyecto simple de análisis de sentimientos.

Instrucciones:

- P reprocesa textos de opiniones
- Calcula similitud con palabras clave positivas y negativas
- Clasifica sentimientos (positivo/negativo/neutral)
- Extrae palabras clave por sentimiento

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from nltk.stem.snowball import SnowballStemmer

stemmer = SnowballStemmer('spanish')

class AnalizadorSentimientos:
    """
    Analizador simple de sentimientos basado en similitud de textos.
    """

    def __init__(self):
        """
        Inicializa con palabras clave de sentimientos.
        """

        self.palabras_positivas = [
            "excelente", "bueno", "increíble", "fantástico", "recomendado",
            "encanta", "perfecto", "maravilloso", "genial", "superó"
        ]

        self.palabras_negativas = [
            "malo", "terrible", "horrible", "defectuoso", "decepción",
            "frustrante", "pésimo", "falló", "mal", "problema"
        ]

        self.palabras_neutras = [
            "normal", "regular", "promedio", "aceptable", "común"
        ]

    def analizar(self, texto):
        """
        Analiza sentimiento de un texto.

        Args:
            texto: Texto a analizar

        Returns:
            Diccionario con análisis
        """
```

Figura 22 Código del Ejercicio 10 Parte 1



```
texto = texto.lower()
texto = re.sub(r'https?\S+|www\S+', '', texto)
texto = re.sub(r'\S+@\S+', '', texto)
texto = re.sub(r'^a-zAÉÍÓÚÑ\s', '', texto)

corpus = [
    texto,
    ".join(self.palabras_positivas),
    ".join(self.palabras_negativas),
    ".join(self.palabras_neutras)
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

sim_positiva = cosine_similarity(X[0:1], X[1:2])[0][0]
sim_negativa = cosine_similarity(X[0:1], X[2:3])[0][0]
sim_neutral = cosine_similarity(X[0:1], X[3:4])[0][0]

similitudes = {
    "positivo": sim_positiva,
    "negativo": sim_negativa,
    "neutral": sim_neutral
}

sentimiento = max(similitudes, key=similitudes.get)
confianza = similitudes[sentimiento]

resultado = {
    'texto': texto,
    'sentimiento': sentimiento,
    'confianza': round(confianza, 3),
    'similitud_positiva': round(sim_positiva, 3),
    'similitud_negativa': round(sim_negativa, 3),
    'similitud_neutral': round(sim_neutral, 3)
}

return resultado
```

Figura 23 Código del Ejercicio 10 Parte 2



```
def extraer_palabras_clave(self, texto):
    texto = texto.lower()
    texto = re.sub(r'http\S+|www\S+', '', texto)
    texto = re.sub(r'\S+@\S+', '', texto)
    texto = re.sub(r'^a-zAÉÍÓÚÑ\s', '', texto)
    palabras = texto.split()

    palabras_positivas_stem = [stemmer.stem(p) for p in self.palabras_positivas]
    palabras_negativas_stem = [stemmer.stem(p) for p in self.palabras_negativas]
    palabras_neutras_stem = [stemmer.stem(p) for p in self.palabras_neutras]

    positivas = [p for p in palabras if stemmer.stem(p) in palabras_positivas_stem]
    negativas = [p for p in palabras if stemmer.stem(p) in palabras_negativas_stem]
    neutras = [p for p in palabras if stemmer.stem(p) in palabras_neutras_stem]

    return {
        "positivas": positivas,
        "negativas": negativas,
        "neutras": neutras
    }

# Textos de prueba
opiniones = [
    "Este producto es excelente y me encanta. Muy recomendado!",
    "Terriblemente malo. No funcionó. Decepcionante y frustrante.",
    "El producto llegó a tiempo. Normal, sin sorpresas.",
    "¡Increíble! Superó todas mis expectativas. Fantástico!",
    "Horrible, defectuoso y de mala calidad. Totalmente decepción."
]

print("Análisis de Sentimientos")
print("-" * 80)

analizador = AnalizadorSentimientos()

print(f"{'Opinión':50} | {'Sentimiento':10} | {'Confianza'}")
print("-" * 80)

for op in opiniones:
    resultado = analizador.analizar(op)
    resumen = op[:50]

    print(f"{'resumen:50'} | {'resultado['sentimiento']:10} | {'resultado['confianza']}'")

    palabras = analizador.extraer_palabras_clave(op)
    print(f"Palabras clave → {palabras}\n")
```

Figura 24 Código del Ejercicio 10 Parte 3

```
Análisis de Sentimientos
=====
Opinión | Sentimiento | Confianza
=====
Este producto es excelente y me encanta. Muy recom | positivo | 0.239
Palabras clave → {'positivas': ['excelente', 'encanta', 'recomendado'], 'negativas': [], 'neutras': []}

Terriblemente malo. No funcionó. Decepcionante y f | negativo | 0.179
Palabras clave → {'positivas': [], 'negativas': ['terriblemente', 'malo', 'decepcionante', 'frustrante'], 'neutras': []}

El producto llegó a tiempo. Normal, sin sorpresas. | neutral | 0.112
Palabras clave → {'positivas': [], 'negativas': [], 'neutras': ['normal']}

¡Increíble! Superó todas mis expectativas. Fantást | positivo | 0.284
Palabras clave → {'positivas': ['increíble', 'superó', 'fantástico'], 'negativas': [], 'neutras': []}

Horrible, defectuoso y de mala calidad. Totalmente | negativo | 0.259
Palabras clave → {'positivas': [], 'negativas': ['horrible', 'defectuoso', 'mala', 'decepción'], 'neutras': []}
```

Figura 25 Resultado del Ejercicio 10



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Conclusiones

El desarrollo del presente taller permitió aplicar de manera práctica los principales conceptos y técnicas del Procesamiento del Lenguaje Natural (NLP), evidenciando su utilidad en el análisis y tratamiento de datos textuales. A través de la ejecución de cada uno de los ejercicios, se logró implementar correctamente procesos como la limpieza y normalización de texto, la tokenización, la eliminación de stopwords y la reducción de palabras mediante stemming y lematización.

Asimismo, se adquirió experiencia en técnicas más avanzadas como el análisis de frecuencia de palabras, la extracción de entidades nombradas (NER) y el cálculo de similitud entre textos, lo que permitió comprender cómo se puede extraer información relevante y medir relaciones semánticas entre diferentes contenidos. La construcción de un pipeline completo de preprocesamiento facilitó la integración de todas estas etapas, demostrando la importancia de estructurar adecuadamente el flujo de procesamiento de datos.

Finalmente, el desarrollo del proyecto integrador de análisis de sentimientos permitió consolidar los conocimientos adquiridos, aplicándolos en un caso práctico que simula situaciones reales. En conjunto, este trabajo fortaleció tanto las habilidades técnicas en programación como la comprensión de cómo las máquinas pueden interpretar el lenguaje humano, resaltando el papel fundamental del NLP en el campo de la inteligencia artificial.

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación





CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

Bibliografía

- [1] S. Russell y P. Norvig, Artificial Intelligence: A Modern Approach, Pearson Education, 1995.
- [2] C. M. Bishop, Pattern Recognition and Machine Learning, Springer-Verlag New York, 2006.
- [3] S. S. Haykin, Neural Networks and Learning Machines, Pearson Education / Prentice Hall., 2008.
- [4] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, O'Reilly Media, 2017.
- [5] I. Goodfellow, Y. Bengio y A. Courville, Deep Learning, MIT Press, 2016.

CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

